

Ship agentic AI, then go under the hood

You already own the engineering half — backend, DevOps, databases, HLD. This is the **AI half**, sequenced so you orchestrate existing models into real products first, and only crack open the model layer when a genuine need calls for it.

Integration (Spring Boot / JS) is deliberately left out — that's your existing strength and slots in around every phase. This tracks only what's new.

The one idea that governs everything: the customization ladder

For any requirement, climb only as far as it takes to solve it. Reaching for a higher rung than you need is the classic — and expensive — mistake.

1	Prompting — instructions + examples to a model	~70% of needs
2	RAG — ground it in your data / docs (facts live here)	your data
3	Tools / Agents — let it act on your systems	do things
4	Fine-tuning — adapt an open model's behavior (LoRA)	style · privacy · scale
5	Train from scratch — build a model from zero	never — \$millions

0

Foundations

UNDERSTAND, DON'T BUILD

~1 WEEK

Get the concepts crisp so nothing downstream feels like magic. This rung is about **understanding** how models work — not building them.

- ☐ Tokens, context windows, temperature, sampling, streaming — the levers of cost, latency & limits
- ☐ Embeddings & vector similarity (cosine) — the single key idea behind RAG
- ☐ Transformer architecture — **conceptual only**: attention, next-token prediction. You use it, you don't build it
- ☐ The customization ladder — prompt vs RAG vs fine-tune vs train, and when each applies
- ☐ Model landscape — frontier (Claude Opus 4.8 / Sonnet 4.6 / Haiku 4.5, GPT) vs open-weight (Llama, Mistral, Qwen, Gemma)
- ☐ Practical Python + environment setup (`venv` , `pip` , notebooks)

MILESTONE Nothing to ship — write a one-page **"how an LLM works"** in your own words. If you can explain tokens, embeddings and the ladder plainly, you're ready.

1

LLM API & Prompt Engineering ~2-3 WEEKS

The core interface between your code and a model. **Everything else is built on this.**

Prompts here are code you write — assembled programmatically on every request.

- ☐ Call an LLM API — message roles (system / user / assistant), params, streaming, multi-turn state
- ☐ Prompt engineering for reliability — system prompts, few-shot examples, delimiter/XML structuring, role & tone
- ☐ Structured output — force JSON / schema your code can parse deterministically
- ☐ Tool / function calling — the foundation of **every** agent
- ☐ Token / cost / latency budgeting; prompt caching; model-per-task selection
- ☐ Treat the model as a flaky dependency — retries, timeouts, output validation, fallbacks

PROJECT **Natural-language** → **action endpoint**. Plain-English request → tool-call → real query against a Postgres DB → returned result.

2

Embeddings, Vector DBs & RAG ~3-4 WEEKS

How an assistant answers questions about **your** data. Facts live here — not in fine-tuning.

This is the "answer things" half of a copilot.

- ☐ Embeddings in depth; choosing an embedding model
- ☐ Vector stores — `pgvector` first (reuse Postgres), then Qdrant / Pinecone; HNSW indexing, metadata filtering
- ☐ Chunking strategies — size, overlap, semantic & structural splitting
- ☐ Full RAG pipeline — ingest → embed → retrieve → ground → cite
- ☐ Hybrid search (semantic + keyword/BM25) & re-ranking — pure vectors alone are rarely enough
- ☐ Grounding & anti-hallucination — citations, and a graceful "I don't know"
- ☐ RAG evaluation — did you retrieve the *right* context?

PROJECT **"Chat with your docs"** over a real corpus. Must cite sources and admit when it doesn't know. Build it raw first — *then* compare a framework.

3

Agents, Tool Use & MCP ~3-4 WEEKS

The heart of a Rovo-style assistant — the "do things in English" engine. An agent is a loop
+ tools + prompts: software engineering, not ML.

- ☐ The agent loop — reason → act → observe → repeat (the ReAct pattern)
- ☐ Multi-tool orchestration — the model choosing among many tools
- ☐ Agentic patterns — planning, reflection, multi-agent — and **when NOT to use an agent**
- ☐ MCP — host / client / server; the primitives (tools, resources, prompts); transports (stdio, HTTP)
- ☐ Build an MCP server exposing real actions from a platform
- ☐ Guardrails — confirm destructive actions, permission scoping, prompt-injection defense
- ☐ Framework — **LangGraph** for stateful agents (+ LangChain / LangSmith); LlamaIndex for retrieval

PROJECT An **MCP server with 3-4 real tools** + an agent that chains them from a single English command ("create a bug, assign it to Sarah, link it to the auth epic").

4

Custom Models — transformers, Fine-tuning & Serving MODEL TRACK · PYTHON

~4-6 WEEKS

Independence from API keys — run and adapt open models yourself. **Reach here only**
when a real need calls for it: privacy, on-prem, cost-at-scale, or a narrow custom behavior.

- ☐ Practical Python + **light PyTorch** — tensors, dtype, device, what a training step is, debugging OOM. Driving-level, *not* from-scratch (~a few days)
- ☐ Hugging Face `transformers` — load & run open models
- ☐ Dataset preparation for fine-tuning — the 80% that actually decides the outcome
- ☐ LoRA / QLoRA fine-tuning with `peft` + `trl` — or Unsloth / Axolotl (easy mode)
- ☐ Quantization for cheaper, faster inference
- ☐ Serving — **vLLM** (fast, OpenAI-compatible) or Ollama; the model becomes an HTTP endpoint your app swaps to
- ☐ Evaluate the fine-tune vs base + prompt — did it actually win?
- ☐ Internalize: fine-tune for **behavior & format**, use **RAG for facts**

PROJECT **Fine-tune an open model with LoRA**, serve it with vLLM, and prove with evals it beats "frontier model + good prompt". Then point your Phase-3 assistant at it — same app, swapped model.

Production AI — Evals, Observability & Safety

YOUR EDGE · START EARLY

ONGOING

AI is non-deterministic, so "does it work?" is a **measurement** problem, not pass/fail. Most teams are weak here — your engineering background makes this your unfair advantage.

- ☐ Evals — build test sets, define metrics, use **LLM-as-judge**, run them like CI to catch regressions
- ☐ Observability / tracing — Langfuse or LangSmith: inputs, outputs, tokens, cost, latency, tool calls
- ☐ Cost & latency optimization — caching, semantic caching, model routing (cheap → escalate)
- ☐ Safety — prompt-injection defense, PII handling, output filtering, tool-permission scoping

PROJECT Add a full **eval suite + tracing + cost dashboard** to your Phase 2/3 assistant.

CAPSTONE

A Rovo-style assistant, end to end

Everything above assembles into one portfolio-defining build: an **agentic assistant embedded in a platform** that —

- takes natural-language commands and **acts** via tools (create / update / link)
- **answers** questions about the platform's data via RAG, with citations
- **respects the user's permissions** and confirms destructive actions
- is **evaluated** and fully **observable** in production
- runs on a frontier model — then swaps to **your self-hosted fine-tuned model** by config

Ship this and you've demonstrated the whole stack — the exact thing the market is scrambling for.

MINDSET SHIFTS

- You **shape probability**, not certainty — and measure it statistically.
- Climb the **cheapest rung** that solves the problem.
- The model is **infrastructure you build on**, not something you build.
- **Facts** → **RAG**. **Behavior** → **fine-tune**.

HOW TO SPEND THE TIME

~3–4 months at ~10 hrs/week gets you job-ready with a real portfolio. Phases 0, 1 & 5 compress hard thanks to your existing skills. Lean on **provider docs & cookbooks** (Anthropic, Hugging Face) over courses — the field moves monthly and courses go stale.

Sequenced for a backend / DevOps engineer · orchestrate first, go deep on demand